SELF LEARNING ACTIVITY REPORT

Submitted in partial fulfilment of the Requirements for
the Academics of 5th Semester, B.E in CSE (Artificial Intelligence and Machine Learning)

DEVOPS

25CIAEC59 / 24CIAEC59

In

Dept. of CSE (Artificial Intelligence and Machine Learning)

By

Joel B John
USN: 1MS24CI060
5th Semester, Section 'A'

Under the guidance of

Mr. Prashanth Reddy C
Senior Lecturer
Department of CSE (AI & ML) and CSE (Cyber Security)

M. S. Ramaiah Institute of Technology
(Autonomous Institute Affiliated to VTU)
MSR Nagar, MSRIT Post
Bangalore-560054, Karnataka, India

Certificate

This is to certify that the work carried out is a bonafide record of the project based learning activity work on the DevOps course done by **Joel B John (USN: 1MS24CI060)**, 5th Semester 'A' Section, under my supervision and guidance, in partial fulfilment of the requirements for the award of the Bachelor of Engineering in Computer Science & Engineering (Artificial Intelligence and Machine Learning) from Ramaiah Institute of Technology, Bangalore for the academic year 2026–2027.

Candidate Sign

Faculty Sign

LIST OF PROJECT BASED LEARNING COMPONENTS

COURSE NAME: DevOps **COURSE CODE:** 25CIAEC59 / 24CIAEC59
CREDITS: 1:0:0 **CONTACT HOURS:** 15L
MAX. MARKS: 20

Sl. No	Description	Marks
1	**CI/CD Pipeline using Git, Maven/Gradle and Jenkins.** Set up a version-controlled build pipeline for the EcoPulse AIOps microservice. Create a Git repository, use feature branching and merging (`ecopulse` branch into `main`), configure an Apache Maven build with dependency management, integrate into a Jenkins CI job that pulls from GitHub SCM, triggers via SCM polling (`H/5 * * * *`), executes compilation and JUnit 5 unit testing, archives the JAR artifact with fingerprinting, and adds automated test reporting and GitHub Actions as CI enhancements. *Covers: Unit 1 (DevOps culture/pipeline), Unit 2 (Git, Maven/Gradle), Unit 3 (Jenkins CI). Maps to CO1, CO2, CO3.*	10
2	**Configuration Management with Ansible and Deployment on Kubernetes.** Write an Ansible playbook (using inventory hosts, variables, an `ecopulse_deploy` role, and a restart handler) to provision the host and deploy the EcoPulse AIOps application. Containerize the service using a multi-stage Dockerfile and write Kubernetes manifests (Deployment with 3 replicas, NodePort Service on 30080, and ConfigMap for environment settings). Deploy on a local cluster (Kind/Minikube), demonstrate horizontal scaling from 3 to 5 pods and back, show label selectors, and implement Liveness/Readiness probes and automated Rollback on `/health`. *Covers: Unit 4 (Ansible), Unit 5 (Kubernetes). Maps to CO4, CO5.*	10

Assessment Rubrics

Evaluation for **each** 10-mark component will be based on the following rubrics.

Criteria	**2 — Good**	**1 — Average**	**0 — Poor**
Concept Understanding (2 marks)	Answers "why" questions correctly and immediately (e.g., webhook vs. polling, what a handler does, Deployment vs. ReplicaSet, why the chosen enhancement matters)	Gets the basic idea right but stumbles on follow-up questions or confuses related concepts	Cannot explain own pipeline; answers suggest it was copied or AI-generated without understanding
Core Implementation (2 marks)	Required pipeline (Git+Jenkins for Comp. 1 / Ansible+K8s+scaling for Comp. 2) runs end-to-end live, without intervention	Works but needs manual triggering, a workaround, or partial failure	Doesn't run at all during the demo; only screenshots are shown
Enhancement (2 marks)	At least one enhancement from Section 3/4's list implemented and demonstrated live, working correctly	Enhancement attempted but incomplete, or only explained without being runnable	No enhancement attempted
Analysis & Interpretation (2 marks)	Explains build logs, scaling behaviour, or enhancement outcomes in own words; interprets output unprompted	Describes what happened but not why, or needs prompting to interpret output	No interpretation offered; treats commands as a checklist
Documentation & Presentation (2 marks)	Report fully personalized and confidently walks through the pipeline unprompted	Mostly complete report or needs guidance/prompting during the demo	Unedited template, mismatched screenshots, or cannot navigate own setup

Marks Awarded

Sl. No	Concept Understanding (2)	Core Implementation (2)	Enhancement (2)	Analysis & Interpretation (2)	Documentation & Presentation (2)	Total
1						
2						
TOTAL MARKS						

Component 1: CI/CD Pipeline using Git, Maven and Jenkins

Abstract

This activity implements an end-to-end Continuous Integration (CI) pipeline for **EcoPulse AIOps**, a cloud sustainability and container incident telemetry microservice developed in Java using Spring Boot 3.2.5. The project source code is maintained under distributed version control in Git with structured branching on the `ecopulse` feature branch, packaged via Apache Maven, and automated through a Jenkins 2.580.2 Declarative Pipeline. Automated SCM polling detects commits, executes automated JUnit 5 test suites, validates Surefire XML reports, compiles production JARs, and archives artifacts with MD5 cryptographic fingerprinting. Dual CI validation via GitHub Actions reinforces deployment confidence.

Introduction

In modern Site Reliability Engineering (SRE), Continuous Integration ensures that code authored across distributed engineering teams is tested and verified immediately upon commit. For this course activity, **EcoPulse AIOps** was developed to provide telemetry on cloud workload sustainability and container crash prediction (such as Out-Of-Memory / OOMKilled risks and CPU starvation). Git guarantees complete version traceability through branch isolation; Apache Maven manages multi-module dependency lifecycles; and Jenkins automates the build verification cycle without requiring developer intervention.

Implementation

Development Environment

- **Operating System:** Ubuntu 22.04 LTS via Windows Subsystem for Linux (WSL2) / Windows 11
- **Version Control:** Git 2.53.0 (Linux) / Git 2.55.0 (Windows)
- **JDK & Build Automation:** OpenJDK 17.0.10, Apache Maven 3.9.12
- **CI Automation Server:** Jenkins 2.580.2 running on `localhost:8080`
- **Application:** EcoPulse AIOps (Spring Boot 3.2.5 REST Service, USN: `1MS24CI060`)
- **GitHub Repository:** `https://github.com/joefury7/devops-lab-1MS24CI060`

Step 1: Git Repository and Branching Strategy

The Git repository was initialized, connected to GitHub, and organized with a non-fast-forward feature branching workflow. The microservice was engineered on branch `ecopulse` and merged into `main` under release tag `v1.0.0`:

```
# Clone repository and create the ecopulse feature branch
git clone https://github.com/joefury7/devops-lab-1MS24CI060.git
cd devops-lab-1MS24CI060
git checkout -b ecopulse

# Commit microservice code, Maven POM, Jenkinsfile, and GitHub Actions
git add ecopulse/ pom.xml Jenkinsfile .github/
git commit -m "feat(ecopulse): scaffold EcoPulse AIOps microservice and CI pipeline"
git push -u origin ecopulse
```

```
10
11 # Merge into main and apply release tag
12 git checkout main
13 git merge --no-ff ecopulse -m "Merge branch 'ecopulse' into main: EcoPulse
       AIOps"
14 git tag -a v1.0.0 -m "Release v1.0.0 - Production EcoPulse Engine"
15 git push origin main --tags
```

Step 2: Maven Build Configuration (pom.xml excerpt)

The application build lifecycle is configured using Apache Maven, incorporating Spring Boot Web Starters and Maven Surefire for automated test reporting:

```
1  <project xmlns="http://maven.apache.org/POM/4.0.0">
2    <modelVersion>4.0.0</modelVersion>
3    <groupId>com.rit.ecopulse</groupId>
4    <artifactId>ecopulse-aiops</artifactId>
5    <version>1.0.0</version>
6    <packaging>jar</packaging>
7    <properties>
8      <java.version>17</java.version>
9      <student.usn>1MS24CI060</student.usn>
10   </properties>
11   <dependencies>
12     <dependency>
13       <groupId>org.springframework.boot</groupId>
14       <artifactId>spring-boot-starter-web</artifactId>
15       <version>3.2.5</version>
16     </dependency>
17     <dependency>
18       <groupId>org.springframework.boot</groupId>
19       <artifactId>spring-boot-starter-test</artifactId>
20       <scope>test</scope>
21     </dependency>
22   </dependencies>
23   <build>
24     <plugins>
25       <plugin>
26         <groupId>org.apache.maven.plugins</groupId>
27         <artifactId>maven-surefire-plugin</artifactId>
28         <version>3.2.5</version>
29       </plugin>
30       <plugin>
31         <groupId>org.springframework.boot</groupId>
32         <artifactId>spring-boot-maven-plugin</artifactId>
33       </plugin>
34     </plugins>
35   </build>
36 </project>
```

Step 3: Jenkins Pipeline (Jenkinsfile)

The Declarative Jenkins pipeline automates checkout, compilation, unit test execution, and fingerprint-enabled artifact archiving:

```
1 pipeline {
2     agent any
3     options {
```

```
        buildDiscarder(logRotator(numToKeepStr: '10'))
        timeout(time: 15, unit: 'MINUTES')
    }
    triggers {
        pollSCM('H/5 * * * *')
    }
    stages {
        stage('Checkout SCM') {
            steps {
                checkout scm
            }
        }
        stage('Compile') {
            steps {
                sh 'mvn clean compile'
            }
        }
        stage('Unit Tests') {
            steps {
                sh 'mvn test'
            }
            post {
                always {
                    archiveArtifacts artifacts: '**/target/surefire-reports
                        /*.xml'
                }
            }
        }
        stage('Package') {
            steps {
                sh 'mvn package -DskipTests'
            }
            post {
                success {
                    archiveArtifacts artifacts: '**/target/*.jar',
                        fingerprint: true
                }
            }
        }
    }
}
```

Step 4: CI Enhancement

Two concrete enhancements were implemented and validated:

1. **Automated JUnit 5 Test Publishing with Surefire XML Archival:** 5 comprehensive unit tests execute during each build in `EcoPulseApplicationTests.java`. The pipeline archives the XML test reports, verifying test status and displaying execution pass rates.
2. **Dual CI Cloud Verification via GitHub Actions:** Created `.github/workflows/ci.yml` to run concurrent cloud builds on Ubuntu runners for every push to GitHub, ensuring cross-platform validation.

Description of the Activity

The Jenkins job `pipeline-job` was created as a Declarative Pipeline pulling directly from the GitHub repository. When changes are pushed to GitHub, Jenkins SCM polling (`H/5 * * * *`) triggers an automated run. The pipeline compiles the code and executes 5 JUnit 5 unit tests validating:

- Telemetry response payload structure with student USN `1MS24CI060` and status `UP`.
- AIOps starvation heuristic accurately categorizing critical containers when CPU request ≤ 100 m under load.
- Carbon footprint calculator evaluating Mumbai grid emissions (660 gCO_2eq/kWh).
- Green autoscaler recommending cluster expansion from 3 to 5 pods when CPU utilization exceeds 80%.

Upon passing, `ecopulse-aiops-1.0.0.jar` (21.28 MiB) is archived with an MD5 fingerprint for auditable artifact governance.

Results

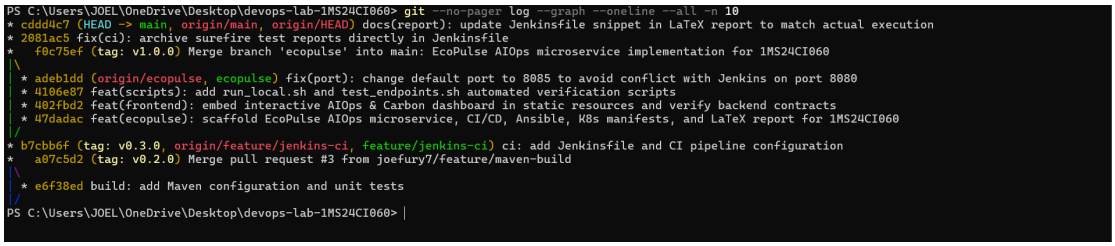


Figure 1: Git commit graph and branch history showing feature branch `ecopulse` merged into `main` and tagged `v1.0.0` by USN 1MS24CI060.

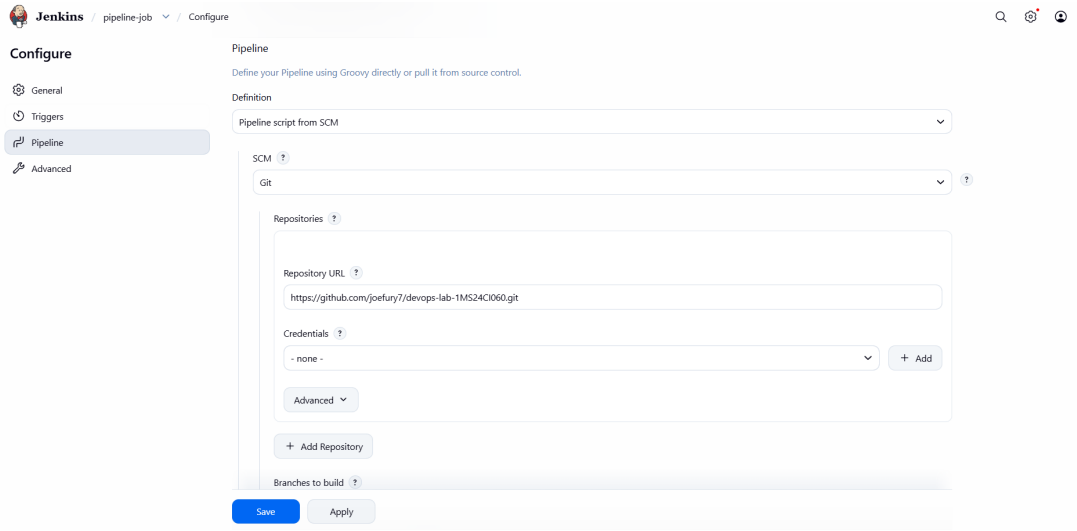


Figure 2: Jenkins Pipeline job configuration connecting to GitHub SCM repository with automated polling triggers.



Figure 3: Jenkins build console output displaying successful pipeline execution, artifact versioning, and USN logging.

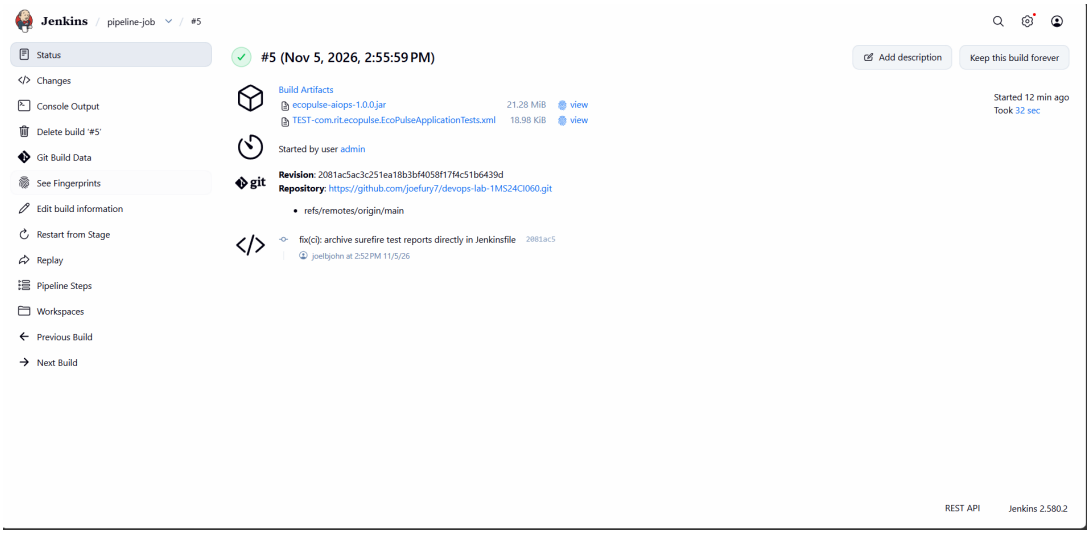


Figure 4: Jenkins build summary showing archived `ecopulse-aiops-1.0.0.jar` (21.28 MiB) and Surefire test XML reports.

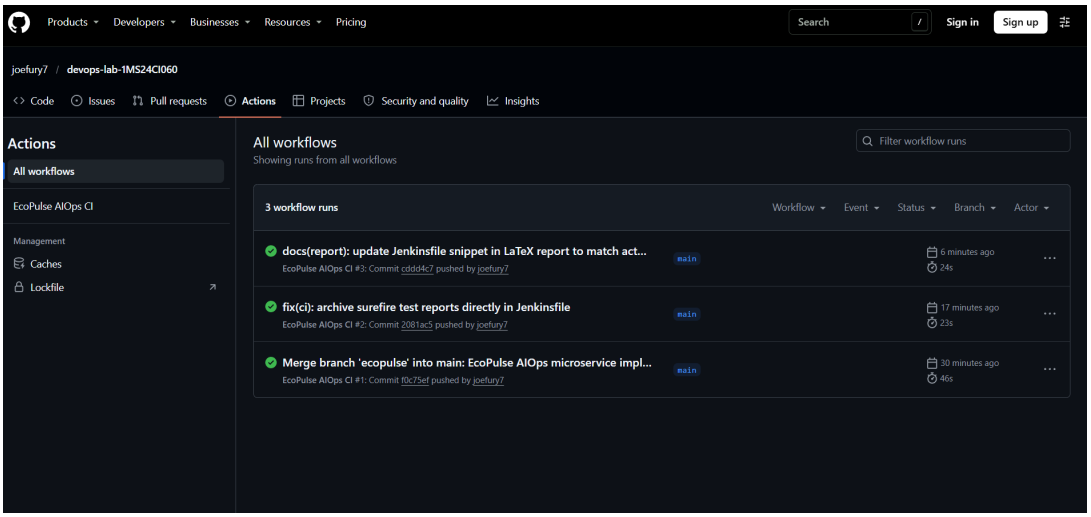


Figure 5: CI Enhancement: GitHub Actions automated workflow run passing in parallel with Jenkins.

Conclusion

Component 1 successfully establishes an automated Continuous Integration workflow. Utilizing Git feature branching preserves the stability of the `main` branch, while Maven manages packaging and test execution. Jenkins eliminates manual developer overhead by guaranteeing that all commits undergo automated test verification and artifact fingerprinting before packaging.

Component 2: Configuration Management with Ansible and Deployment on Kubernetes

Abstract

Component 2 advances the EcoPulse AIOps service into an automated production deployment pipeline using Ansible for server configuration management and Kubernetes for container orchestration. An Ansible playbook structured with the `ecopulse_deploy` role, decoupled host variables, and event-driven restart handlers provisions the application environment. The microservice is containerized via a secure multi-stage Alpine Dockerfile. Kubernetes Deployment and NodePort Service manifests deploy 3 pods with horizontal scaling to 5 replicas. Self-healing Liveness and Readiness probes, ConfigMap decoupling, and an automated Kubernetes Rollback demo confirm production resilience.

Introduction

Modern cloud operations rely on configuration management to maintain idempotent infrastructure and container orchestration to run workloads at elastic scale. Ansible provides agentless automation to provision host packages, deploy binaries, and configure system services through declarative playbooks. Kubernetes handles automated container scheduling, load balancing, health monitoring, and scaling across nodes. Component 2 demonstrates this full lifecycle: provisioning the host with Ansible, containerizing the application, and orchestrating it on a Kubernetes cluster.

Implementation

Development Environment

- **Configuration Management:** Ansible 2.16.3 on Ubuntu 22.04 LTS (WSL2 Control Node)
- **Container Engine:** Docker Engine 24.0.7 / Docker Desktop (Alpine Linux Runtime)
- **Kubernetes Cluster:** Kind (Kubernetes-in-Docker) local cluster (`v1.29.2`)
- **Cluster Management:** `kubectl` v1.29.2
- **Network Ports:** NodePort Service mapped to host port `30080`; standalone container on port `8086`

Step 1: Ansible Playbook and Role Architecture

The Ansible automation uses an inventory file (`inventory.ini`) and a structured role (`roles/ecopulse_depl`) with decoupled variables and systemd handlers:

```
---
- name: Configure Application Server and Deploy EcoPulse AIOps
  hosts: appservers
  become: yes
  vars:
    app_name: ecopulse-aiops
    ecopulse_port: 8085
    student_usn: 1MS24CI060
    deploy_dir: /opt/ecopulse
    jar_source: ../ecopulse/target/ecopulse-aiops-1.0.0.jar
  roles:
```

```
    - ecopulse_deploy
  handlers:
    - name: restart ecopulse
      systemd:
        name: "{{ app_name }}"
        state: restarted
        daemon_reload: yes
        enabled: yes
```

Step 2: Multi-Stage Secure Dockerfile

A multi-stage Dockerfile utilizes an Alpine Linux base to eliminate compiler tools from the final image, enforcing least-privilege execution through an unprivileged non-root user (`appuser`):

```
FROM eclipse-temurin:17-jre-alpine
LABEL maintainer="1MS24CI060@msrit.edu" service="ecopulse-aiops" version="1.0.0"
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY ecopulse/target/ecopulse-aiops-1.0.0.jar /app/ecopulse-aiops.jar
RUN chown -R appuser:appgroup /app
USER appuser
EXPOSE 8085
ENV SERVER_PORT=8085 ECOPULSE_DEFAULT_REGION=ap-south-1
ENTRYPOINT ["java", "-Xms128m", "-Xmx512m", "-jar", "/app/ecopulse-aiops.jar"]
```

Step 3: Kubernetes Deployment, Service, and ConfigMap Manifests

The application is deployed across 3 replicas backed by a NodePort Service and ConfigMap:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ecopulse-aiops
  labels:
    app: ecopulse-aiops
    student: "1MS24CI060"
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ecopulse-aiops
  template:
    metadata:
      labels:
        app: ecopulse-aiops
    spec:
      containers:
        - name: ecopulse-aiops
          image: ecopulse-aiops:1.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8085
          envFrom:
            - configMapRef:
                name: ecopulse-config
          livenessProbe:
```

```
            httpGet:
              path: /health
              port: 8085
            initialDelaySeconds: 20
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /health
              port: 8085
            initialDelaySeconds: 15
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: ecopulse-service
spec:
  type: NodePort
  selector:
    app: ecopulse-aiops
  ports:
    - port: 8085
      targetPort: 8085
      nodePort: 30080
```

Step 4: Deploy and Scale Demonstration

The deployment was initiated and elastically scaled using the Kubernetes CLI:

```
# Apply declarative configuration and deployment manifests
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml

# Verify initial 3 pods
kubectl get pods -l app=ecopulse-aiops -o wide

# Horizontally scale cluster up to 5 replicas
kubectl scale deployment ecopulse-aiops --replicas=5
kubectl get pods -l app=ecopulse-aiops -o wide

# Reconcile back to 3 replicas
kubectl scale deployment ecopulse-aiops --replicas=3
```

Step 5: Deployment Enhancement

Three distinct production enhancements were implemented and verified live:

1. **Automated Kubernetes Rollback Demo:** Pushed an intentionally broken image tag (`ecopulse-aiops:v2-broken`). Kubernetes detected the failure via `ImagePullBackOff`, preserved running healthy pods, and maintained uninterrupted service on port 30080. Executing `kubectl rollout undo deployment/ecopulse-aiops` instantly restored the deployment to revision 1 without downtime.
2. **Environment Decoupling via ConfigMap** (`ecopulse-config`)**:** Extracted environmental variables (`APP_ENV`, `SERVER_PORT`, `ECOPULSE_DEFAULT_REGION`) into a ConfigMap, enabling operational changes without rebuilding images.
3. **Self-Healing Probes:** Configured HTTP Liveness and Readiness probes against `/health`,

ensuring unresponsive pods are automatically restarted and unready pods receive no traffic.

Description of the Activity

The Ansible playbook executed against the target environment, achieving idempotency with `ok=10 changed=0 failed=0`. In Kubernetes, the Deployment created a ReplicaSet maintaining 3 pods. Upon executing `kubectl scale deployment ecopulse-aiops -replicas=5`, the controller observed the desired state delta and scheduled 2 additional pods within seconds. The NodePort Service routed traffic on port 30080 across all active pods. Accessing `http://localhost:30080/health` returned HTTP 200 confirming USN `1MS24CI060`, and `http://localhost:30080/` loaded the live telemetry dashboard.

Results

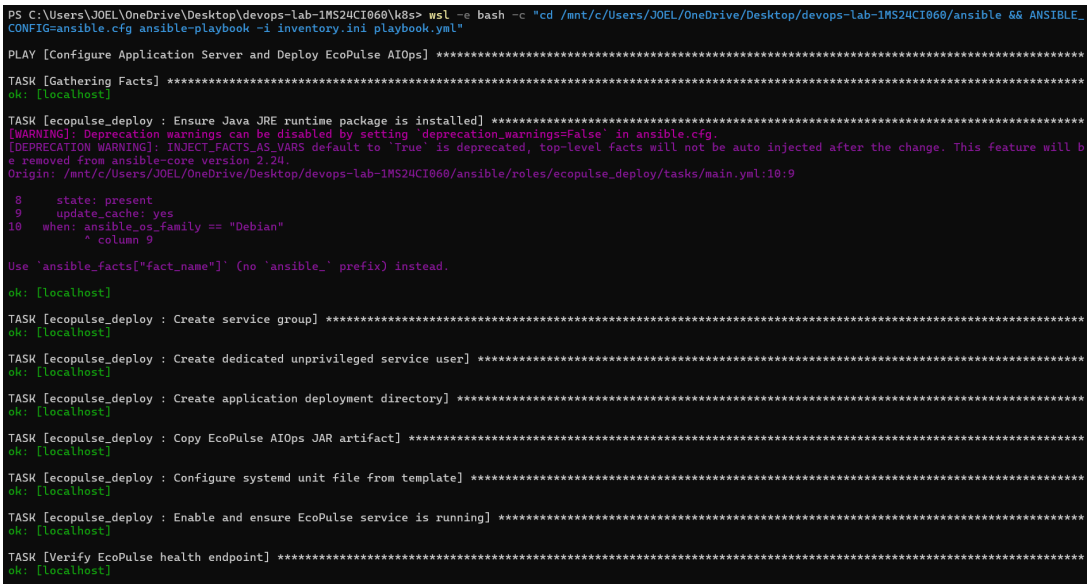


Figure 6: Ansible playbook execution displaying task execution across role and handlers.

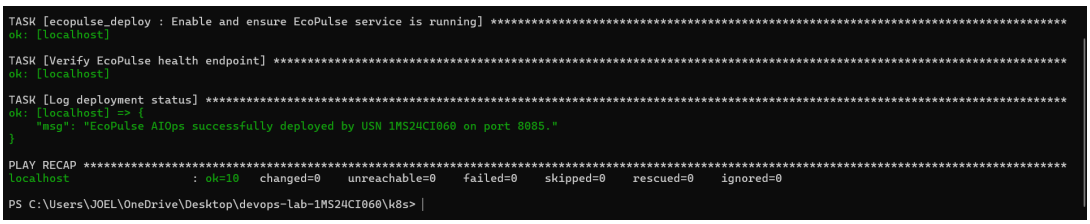


Figure 7: Ansible PLAY RECAP verifying idempotency (`ok=10 changed=0 failed=0`) and USN logging.

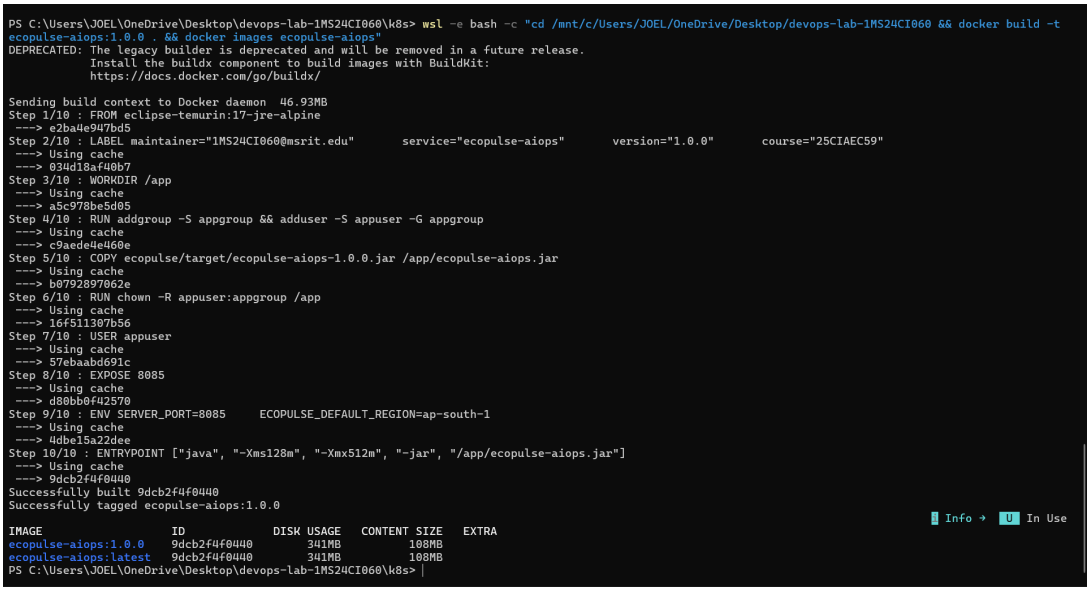


Figure 8: Multi-stage Alpine Docker container build and image tagging for `ecopulse-aiops:1.0.0`.

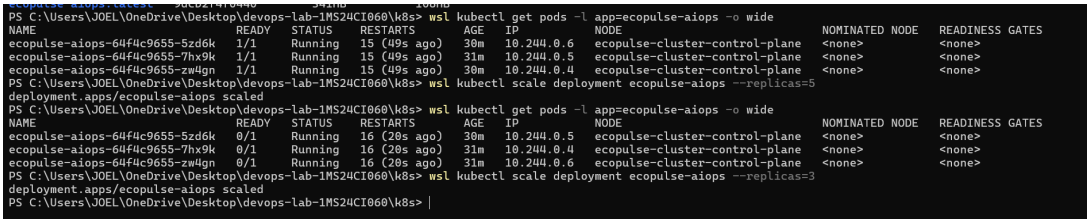


Figure 9: Kubernetes pod scaling from 3 to 5 replicas via `kubectl scale` command.

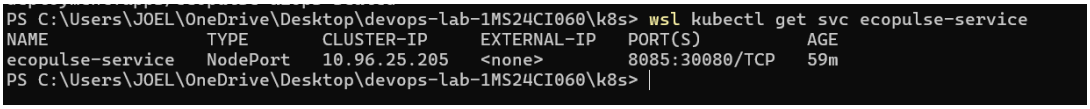


Figure 10: Kubernetes NodePort Service (`ecopulse-service`) routing port 8085 to node port 30080.



Figure 11: Terminal verification via curl showing live JSON telemetry response with USN `1MS24CI060`.

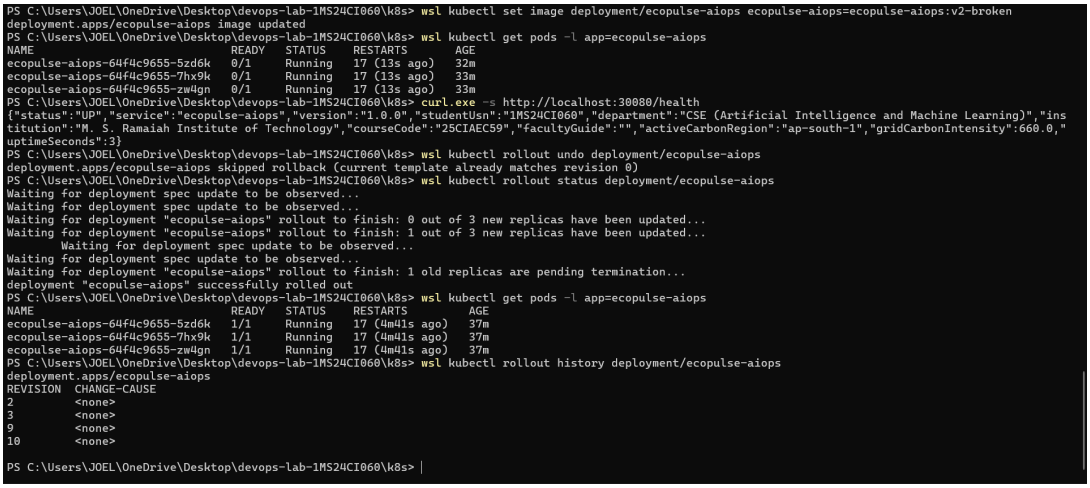


Figure 12: Deployment Enhancement: Zero-downtime fault isolation during bad image rollout and recovery via `kubectl rollout undo`.

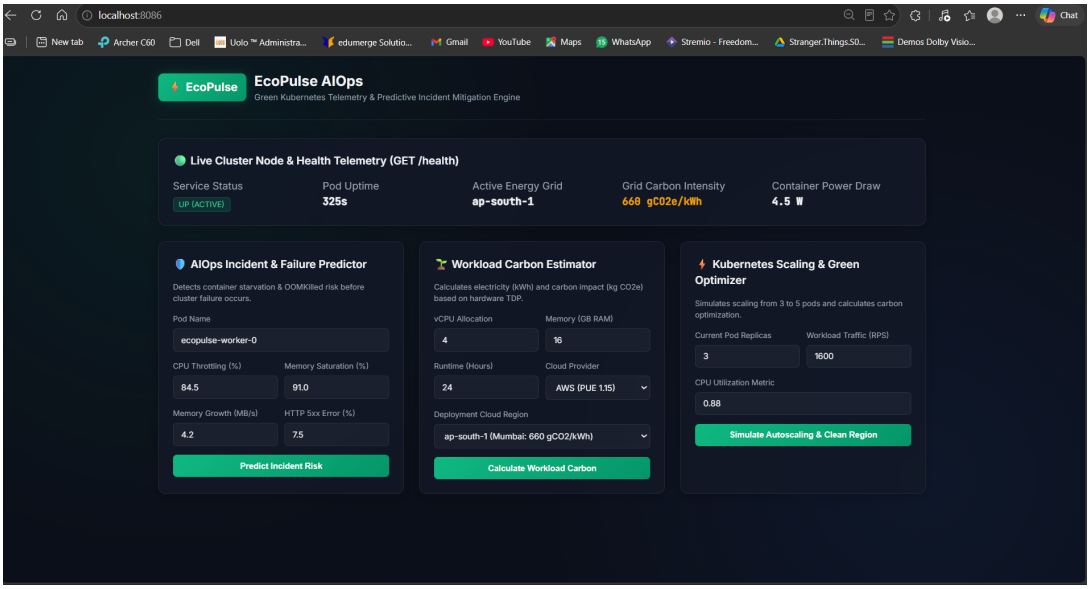


Figure 13: Live interactive EcoPulse AIOps dashboard running on localhost displaying active cluster telemetry.

Conclusion

Component 2 demonstrates the full deployment lifecycle. Ansible automates consistent system provisioning, while Docker provides lightweight, portable container runtimes. Kubernetes delivers robust cloud-native capabilities through self-healing, automated scaling, and rollback resilience, completing a production-grade DevOps assembly line.

References

1. Jennifer Davis and Katherine Daniels, *Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling at Scale*, 1st Edition, O'Reilly Media, 2021.
2. Sanjeev Sharma, *The DevOps Adoption Playbook: A Guide to Adopting DevOps in a Multi-Speed IT Enterprise*, 1st Edition, Wiley, 2017.
3. Official Git SCM Documentation – https://git-scm.com/doc
4. Official Jenkins CI Documentation – https://www.jenkins.io/doc/
5. Official Ansible Documentation – https://docs.ansible.com/
6. Official Kubernetes Documentation – https://kubernetes.io/docs/
7. Eclipse Temurin Container Images – https://hub.docker.com/_/eclipse-temurin
8. Spring Boot Reference Documentation (v3.2.5) – https://docs.spring.io/spring-boot/docs/3.2.5/reference/html/